TIME TABLE GENERATOR

A timetable generator created for generating random timetable keeping in perspective of teachers work hours and taking care of repetition and overlapping.

MADE BY: BRYAN

INDIAN SCHOOL AL MAABELA

(ISO 9001:2015 CERTIFIED INSTITUTION)

COMPUTER SCIENCE PROJECT

2024-2025

TIME TABLE GENERATOR

Submitted By : Bryan Joe
Class : XII A
Registration No : _____

INDIAN SCHOOL AL MAABELA

CERTIFICATE

It is hereby certified that Master/~~Miss~~ _____ *Bryan Joe* _____ *of class XII A has carried out the necessary project work in Computer Science (083) as per the syllabus prescribed by the* ***C****entral* ***B****oard of* ***S****econdary* ***E****ducation, New Delhi, for the year 2024-2025.*

Internal Examiner **External Examiner**

Date ***Examiner No:***

School Seal **Principal**

ACKNOWLEDGEMENT

*On the successful completion of my **Computer Science** project, first of all I thank the **almighty GOD** who gave me the **strength** and **courage** to accomplish this **task**.*

*I extend my sincere thanks to this esteemed **institution**, **Indian school Al Maabela**, for nurturing me all these years and molding me into a good **academic** and **culture citizen.***

*I place on **record** my heartfelt gratitude to our beloved **Principal Mr. Parveen Kumar** and **Vice Principal Mrs. Savita Saluja** for creating a conducive learning environment and being an **inspiration**, which motivated me to fulfill my **aspirations** and achieve my **goals**.*

*I am deeply indebted to my **Computer Science** project guide **Mrs. Deepa C M** under whose **vital suggestions** and **patient guidance** I could complete this **project** most **successfully**.*

*On a moral personal note, my deepest appreciation to my loving **parents** and **friends**, who put their trust in me and provided me with unceasing **encouragement** and **support**.*

November 2024

Index

Abstract

In most schools, timetable scheduling is a complex and time-consuming process. Traditionally, this task is performed manually or through simple spreadsheet tools, which can lead to inefficiencies and frequent conflicts. Manual scheduling requires significant attention to detail and a thorough understanding of each teacher's availability, subject expertise, and classroom needs. The complexity increases with the number of classes, subjects, and teachers involved, often resulting in overlaps or gaps in schedules. Inaccuracies not only disrupt teaching but also lead to time management challenges for both teachers and students.

To address these issues, an automated timetable scheduling system can offer significant benefits. An effective automated solution minimizes the risk of conflicts, reduces administrative burden, and increases overall scheduling efficiency. Furthermore, it allows for real-time updates, making it easier to adapt to changes such as teacher absences, schedule revisions, or new class requirements. With an intuitive interface, school administrators can manage timetables seamlessly and ensure the efficient allocation of teaching resources.

Objective:

The primary objective of this project is to design and develop a School Time Table Generator, a web-based application that automates the scheduling process by assigning periods to teachers based on predefined criteria such as availability, subject expertise, and class requirements. The application is intended to provide:

- Automated scheduling of teaching periods to ensure optimal utilization of teacher resources.
- Conflict detection and resolution to avoid overlapping assignments and conflicting schedules.
- A flexible user interface that allows administrators to make manual adjustments as needed.

This project aims to simplify the management of school timetables by providing an efficient, reliable, and user-friendly system that can adapt to different school requirements. By utilizing SQL for database management, Python for backend logic, and Flask for the user interface, this application will deliver a complete scheduling solution tailored to school environments.

Scope:

The School Time Table Generator focuses on the creation and management of timetables for teachers and classes within a school. This project will provide the following core functionalities:

- **Teacher and Class Management**: Define teacher availability, subject assignments, and class requirements.
- **Automated Scheduling**: Generate timetables based on teacher availability and subject requirements, ensuring each class receives its necessary lessons without conflicts.
- **Conflict Resolution**: Detect potential scheduling conflicts and provide mechanisms for automatic or manual adjustments.
- **User Interface for Timetable Management**: Offer a web-based interface that allows administrators to view and manage timetables with real-time updates.

The system is designed to be flexible and adaptable for schools of different sizes and requirements. While the primary focus is on creating a basic scheduling framework, this application can be expanded in future versions to include additional functionalities such as holiday management, support for special events, and a mobile-friendly interface. However, the current version will not support advanced scheduling algorithms for holiday handling or custom events, which are reserved for potential future development.

History of Python

Python was created by Guido van Rossum in the late 1980s and first released in 1991. Designed as a high-level programming language emphasizing readability and simplicity, Python quickly gained popularity due to its clear syntax and versatile nature. Initially intended for educational purposes, Python evolved into a powerful tool for a wide range of applications, including web development, data science, automation, artificial intelligence, and more. Its extensive libraries and active community have made Python one of the most widely used programming languages globally.

Applications of Python

Python's versatility allows it to be used across various fields:

- **Web Development**: Frameworks like Django and Flask enable robust web applications.
- **Data Science and Machine Learning**: Libraries like Pandas, NumPy, and Scikit-Learn support data analysis, while TensorFlow and PyTorch facilitate machine learning applications.
- **Automation and Scripting**: Python simplifies repetitive tasks, making it popular for task automation and scripting.
- **Education**: Python's beginner-friendly syntax makes it an excellent choice for teaching programming concepts.
- **Game Development**: Libraries such as Pygame enable the creation of 2D games.
- **Artificial Intelligence and Robotics**: Python's support for AI libraries helps develop intelligent applications and control robotic systems.

Given its components, the School Time Table Generator project can be categorized under multiple applications of Python:

Data Management:

The project is fundamentally a data-driven application that manages and organizes complex datasets (teachers, subjects, schedules) and handles data integrity, consistency, and retrieval through structured SQL databases. Python is used here to manage these interactions efficiently.

Automation and Scripting:

The timetable generator automates the scheduling process by generating timetables based on predefined rules, minimizing manual work. Python's scripting capabilities drive this automation, making the application valuable for routine and repetitive scheduling tasks.

Web Development:

With Flask as the framework, the project includes a web-based interface where administrators can view, update, and manage the timetable. Python's Flask framework is commonly used in web development to build interactive, accessible applications.

Database - Basic Concepts

Database: A database is a structured system that allows users to store, manage, and access data electronically. It is designed to handle large amounts of information and ensure data consistency, accuracy, and security.

Types of Databases

- **Relational Databases**: Store data in tables with rows and columns, with relationships between tables. Examples include MySQL, PostgreSQL, and SQLite.
- **Non-Relational Databases (NoSQL)**: Store data in formats like documents, key-value pairs, or graphs. Examples include MongoDB, Cassandra, and Firebase.
- **Hierarchical Databases**: Use a tree-like structure to represent data.
- **Object-Oriented Databases**: Store data in the form of objects, similar to object-oriented programming concepts.

Database Management System (DBMS)

A **DBMS** is software that interacts with users and applications to manage the database. It ensures data consistency, security, and efficient handling of large datasets. Examples include:

- **MySQL**: Open-source relational database management system.
- **Oracle DB**: Proprietary DBMS with advanced features for large-scale systems.
- **Microsoft SQL Server**: A relational database system for enterprise-level applications.

Key Database Concepts

- **Tables**: The fundamental unit of storage in relational databases. Tables consist of rows (records) and columns (attributes).
- **Primary Key**: A unique identifier for each record in a table.
- **Foreign Key**: A column in one table that establishes a link to the primary key of another table.
- **Relationships**: Define how tables are connected (e.g., one-to-one, one-to-many, many-to-many).
- **Normalization**: The process of organizing data to reduce redundancy and improve data integrity.
- **Indexes**: Structures that improve the speed of data retrieval operations.

SQL (Structured Query Language)

SQL is the language used to interact with relational databases. Key SQL operations include:

- **Data Definition Language (DDL):** Commands like CREATE, ALTER, and DROP for defining database structure.
- **Data Manipulation Language (DML):** Commands like SELECT, INSERT, UPDATE, and DELETE for managing data.
- **Data Control Language (DCL):** Commands like GRANT and REVOKE for controlling access.
- **Transaction Control Language (TCL):** Commands like COMMIT and ROLLBACK for managing transactions.

Database Connectivity

- The School Time Table Generator project uses mysql-connector to connect the Python application with a MySQL database. This library allows the system to execute SQL queries for managing and retrieving data, such as teacher availability, class schedules, and subject assignments. The connection is established using credentials like host, username, password, and database name, ensuring secure communication between the application and the database.
- With **mysql-connector**, the application can insert, update, and query the database to generate timetables and resolve scheduling conflicts. The library helps automate the timetable generation process by fetching the necessary data and ensuring there are no overlaps in the teacher's schedule, providing an efficient and reliable solution for database management.

Steps for SQL Connectivity with Python:

Install the Required Library

Install the mysql-connector library to enable Python-to-MySQL communication:

```
pip install mysql-connector-python
```

Import the Library

Import the mysql.connector module to access its database connectivity features:

```
import mysql.connector
```

Establish a Connection

Use the mysql.connector.connect() method to establish a connection to the MySQL

```
connection = mysql.connector.connect(
    host="localhost",          # Database server address
    user="your_username",      # Database username
    password="your_password",  # Database password
    database="your_database"   # Database name
)
```

Create a Cursor Object

The cursor object allows execution of SQL commands:

```
cursor = connection.cursor()
```

Write and Execute SQL Queries

Write and execute SQL queries using the cursor.execute() method. For example:

```
query = "SELECT * FROM your_table_name"
cursor.execute(query)
```

Fetch Results (for Retrieval Queries)

Fetch results from the query using fetchone(), fetchall(), or fetchmany():

```
results = cursor.fetchall()
for row in results:
    print(row)
```

Commit Changes (for Insert, Update, or Delete Queries)

For database modification queries, commit the changes using connection.commit():

```
connection.commit()
```

Close the Cursor and Connection

Free up resources by closing the cursor and connection:

```
cursor.close()
connection.close()
```

Implementation

- **Database Design**: SQL tables like Teacher, Class, Period, Subject, and Schedule, with relationships defined using primary and foreign keys.
- **Backend Logic**: Python modules handle period allocation, conflict detection, and fair distribution of teaching load.
- **Flask App Setup**: Flask manages web routes and views, with routes like login, dashboard, and generate_schedule.
- **SQL Queries**: Sample SQL queries include:
- **Insert**: Adding new teachers or classes.
- **Select**: Retrieving schedules for a teacher.
- **Update**: Modifying schedules or teacher availability.

Key Features

- **Automated Period Allocation:** Automatically assigns periods based on teacher availability, subject requirements, and institutional rules, reducing manual work significantly.
- **Export to CSV:** The timetable can be exported to a CSV file, which is easily shareable and editable for record-keeping or further adjustments.
- **Conflict Resolution:** Prevents scheduling conflicts by ensuring unique assignments, enhancing accuracy and fairness.
- **Timetable Customization:** Manual adjustments are allowed, providing flexibility to accommodate sudden changes or special requests.
- **Real-Time Management:** Changes to schedules, teacher availability, or subject requirements are updated instantly in the system, ensuring up-to-date timetables.

Testing

- **Unit Testing**: Modules are individually tested, such as scheduling logic and database interactions.
- **Integration Testing**: Ensures seamless interaction between frontend and backend.
- **User Testing**: User feedback from teachers and staff to refine functionality and ease of use.

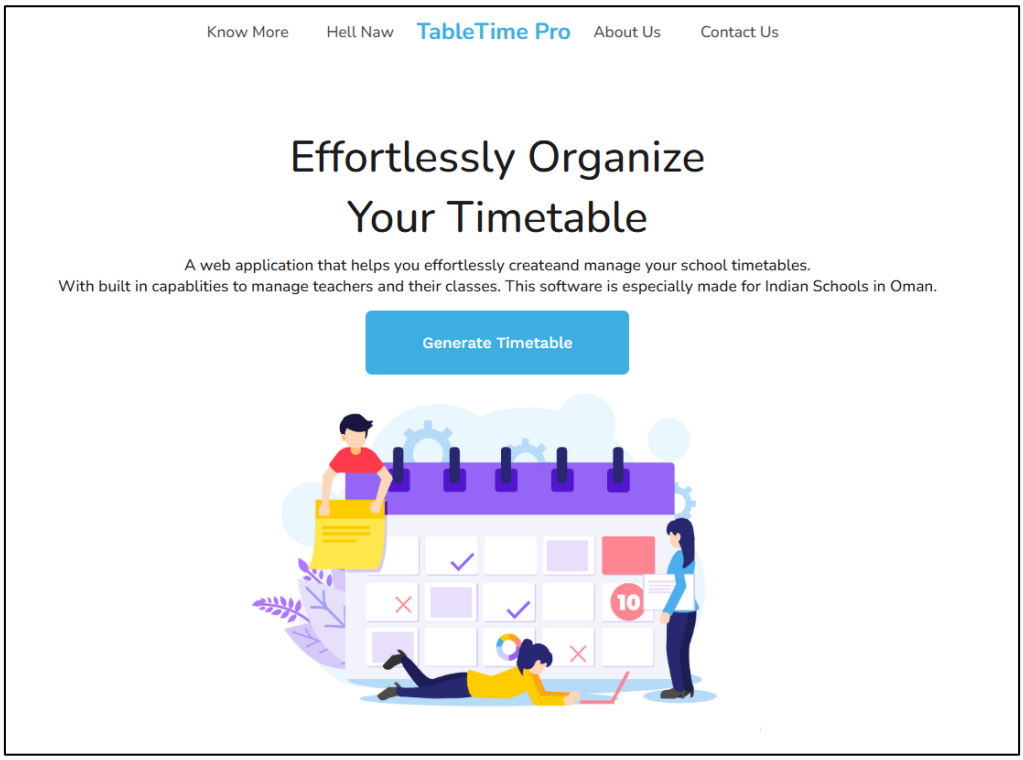
Results and Discussion

- **Results**: Timetable generator produces accurate schedules with screenshots showcasing functionalities.
- **Performance Analysis**: Efficient with minimal delays in scheduling large datasets.
- **Real-Life Applications:** The tool saves administrative time, reduces manual effort, and ensures error-free schedules. Its adaptability to real-world changes, such as teacher absences or class reschedules, enhances operational efficiency in educational institutions.
- **Limitations**: Future features could address holiday schedules and unexpected changes.

Conclusion

This School Time Table Generator successfully automates and streamlines scheduling processes, minimizing errors and reducing the time needed for timetable management. Future enhancements could include mobile support, holiday scheduling, and improved conflict resolution.

Interface and Working



WEEKLY TIMETABLE

11A

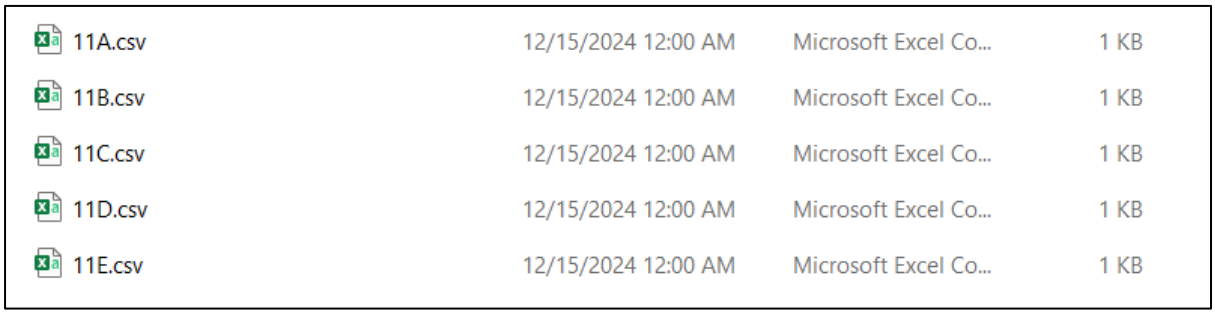
11A	First	Second	Third	Fourth	Fifth	Sixth	Seventh	Eighth
Sun	Eng	Skill	Phy	Chem	Prac	Prac	Comp	Math
Mon	Eng	Math	Math	P_Comp	P_Comp	Phy	Skill	Comp
Tue	Comp	Phy	Math	Eng	PE	Eng	Math	Chem
Wed	Math	Comp	Comp	Skill	Chem	Phy	Phy	Eng
Thu	Eng	Chem	Comp	Chem	Phy	Comp	Math	Math

11B

11B	First	Second	Third	Fourth	Fifth	Sixth	Seventh	Eighth
Sun	Phy	Skill	Comp	Math	Comp	Eng	P_Bio	P_Bio
Mon	Comp	Prac	Prac	Chem	Math	Eng	Skill	Phy
Tue	Chem	Eng	Phy	Comp	PE	Math	Math	Eng
Wed	Math	Comp	Eng	Skill	Chem	Phy	Comp	Phy
Thu	P_Comp	P_Comp	Math	Chem	Phy	Eng	Comp	Chem

./Classes

./Classes/11Grade



./Classes/11Grade/11A.csv

	A	B	C	D	E	F	G	H	I
1	11A	1	2	3	4	5	6	7	8
2	Sun	Phy	Chem	Math	Eng	Eng	Skill	Math	Comp
3	Mon	Math	Skill	PE	Comp	Phy	Math	Chem	Eng
4	Tue	Comp	Math	Math	Chem	Prac	Prac	Phy	Eng
5	Wed	Phy	Chem	Math	P_Comp	P_Comp	Eng	Comp	Phy
6	Thu	Math	Math	Eng	Skill	Phy	Comp	Phy	Chem

SQL Database

ID	ClassName	Day	Period	Subject
1	11A	Mon	1	Phy
2	11A	Mon	2	Eng
3	11A	Mon	3	Chem
4	11A	Mon	4	Phy
5	11A	Mon	5	Comp
6	11A	Mon	6	Eng
7	11A	Mon	7	Chem
8	11A	Mon	8	Math
9	11A	Tue	1	Chem
10	11A	Tue	2	Phy
11	11A	Tue	3	Skill
12	11A	Tue	4	Math
13	11A	Tue	5	Eng

fieldName	fieldValue
noOfDays	5
periodsPerDay	8
maxClassesInGround	2

```
1 •   SELECT * FROM TeacherDetails WHERE subject = 'Math';
```

Result Grid | Filter Rows: | Edit: | Export/Imp

id	name	subject	classes
1	Karuna	Math	11A, 12A
2	Reshma	Math	11B, 12B
3	Moses	Math	11C, 12C
NULL	NULL	NULL	NULL

```
1 •  SELECT * FROM PEPeriods;
```

Result Grid | Filter Rows: | Edit: | Export/Impo

id	teacher	subject	classes	periodsPerDay
1	Karthik	PE	[["11A","11B"],["11C","11D","11E"]]	5
2	Ameen	PE	[["12A","12B"],["12C","12D","12E"]]	5
NULL	NULL	NULL	NULL	NULL

```
1 •  SELECT * FROM SkillSubjectPeriods;
2
```

Result Grid | Filter Rows: | Edit: | Export/Import:

id	teachers	subject	grade	periodsPerWeek
1	["Ameen","Ashley","Kavitha","Ruskin","Karthik"]	Skill	11	3
2	["Ameen","Ashley","Kavitha","Ruskin","Karthik"]	Skill	12	3
NULL	NULL	NULL	NULL	NULL

```
1 •  SELECT * FROM PracticalPeriods;
2
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell

id	name	lab	subjects	classes	continu
1	P_Comp	["Comp"]	["Comp"]	[(["11A"],["Deepa"]),(["11B","11C"],["Shibu","R...	2
2	P_Bio	["Bio"]	["Bio"]	[(["11B"],["Ashwathy"]),(["11C"],["Vijitha"])]	2
3	Prac	["Chem","Phy"]	["Chem","Phy"]	[(["11A"],["Shruthi","Anuradha"]),(["11B"],["Sh...	2
4	P_Comp	["Comp"]	["Comp"]	[(["12A"],["Deepa"]),(["12B","12C"],["Shibu","R...	2
5	P_Bio	["Bio"]	["Bio"]	[(["12B"],["Ashwathy"]),(["12C"],["Vijitha"])]	2
6	Prac	["Chem","Phy"]	["Chem","Phy"]	[(["12A"],["Shruthi","Vidhya"]),(["12B"],["Shafe...	2
NULL	NULL	NULL	NULL	NULL	NULL

```
1 •  SELECT * FROM Periods;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap

id	subject	classes	maxPerDay	noPerWeek	repeatPerWeek
1	Math	["11A","11B","11C","12A","12B","12C"]	[2,1]	7	1
2	Chem	["11A","11B","11C","12A","12B","12C"]	[2,1]	7	1
3	Phy	["11A","11B","11C","12A","12B","12C"]	[2,1]	6	1
4	Comp	["11A","11B","11C","12A","12B","12C"]	[2,1]	6	1
5	Eng	["11A","11B","11C","12A","12B","12C"]	[2,1]	6	1
NULL	NULL	NULL	NULL	NULL	NULL

Code

main.py

```
import random as rand
import data
from timetable import *

table = Timetable(8,5)

table.generateTimetable()
# table.printTimetable()
table.updateSQLData()

table.saveAsCSV()
```

app.py

```
from flask import Flask, render_template
import main
app = Flask(__name__)

timetable = main.table.getData()

@app.route('/')
def home():
    return render_template("home.html",data = timetable,table = main.table)

if __name__ == '__main__':
    app.run(host="0.0.0.0", port=8000, debug=True)
```

data.py

```python
import mysql.connector as sql
import csv
import os
import copy

class MySqlConnector:
    def __init__(self):
        self.host = 'localhost'
        self.user = 'root'
        self.password = 'Lmao'

        self.DB = sql.connect(
            host=self.host,
            user=self.user,
            password=self.password,
            database='timetable'
        )
        self.cursor = self.DB.cursor()

    def getConfig(self):
        self.cursor.execute("SELECT fieldName, fieldValue FROM config")
        result = self.cursor.fetchall()
        config_data = {row[0]: int(row[1]) for row in result}
        return config_data

    def updateOutput(self,data):
        self.cursor.execute('''DROP TABLE ClassSchedule''')
        self.cursor.execute('''CREATE TABLE IF NOT EXISTS ClassSchedule(
            ID INTEGER PRIMARY KEY AUTO_INCREMENT,
            ClassName TEXT NOT NULL,
            Day TEXT NOT NULL,
            Period INTEGER NOT NULL,
            Subject TEXT NOT NULL
        )''')
        self.DB.commit()

        days = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri']
        index = 1
        for class_name, weekly_schedule in data.items():
            for day_index, day_schedule in enumerate(weekly_schedule):
                day = days[day_index]
                for period, subject in enumerate(day_schedule, start=1):
                    # print(f"INSERT INTO ClassSchedule VALUES ({index},'{class_name}', '{day}',
{period}, '{subject if subject else "idk"}')")
                    self.cursor.execute(f"INSERT INTO ClassSchedule VALUES ({index},'{class_name}',
'{day}', {period}, '{subject if subject else "idk"}')")
                    index += 1
        self.DB.commit()
        self.DB.close()
```

```
class Data(MySqlConnector):
    def __init__(self,periodsPerDay,noOfDays):
        super().__init__()
        cofigData = self.getConfig()

        self.noOfDays = cofigData['noOfDays']
        self.periodsPerDay = cofigData['periodsPerDay']

        self.maxClassesInGround = cofigData['maxClassesInGround']

        self.days = ['Sun','Mon','Tue','Wed','Thu','Fri','Sat']
        self.ordinals =
['First','Second','Third','Fourth','Fifth','Sixth','Seventh','Eighth','Ninth','Tenth']

        # Should get rid of this
        self.emptyPossiblePeriods = [x for x in range(0,periodsPerDay)]
        self.emptyPossibleDays = [x for x in range(0,self.noOfDays)]

        self.possibleIndexes = {}

        # List of all classes in the school
        self.classes = [
            {'grade':11,'maxSection':'E'},
            {'grade':12,'maxSection':'E'}
        ]

        self.teachers =
['Karuna','Reshma','Moses','Shafeela','Shruthi','Krithika','Vidhya','Anuradha','Lida','Vijitha','Ashwa
thy','Deepa','Judith','Shibu','Anitha','Irine','Ameen','Karthik','Ashley','Kavitha','Ruskin']
        self.labs = ['Chem','Phy','Bio','Comp","SComp"]

        # Data for creating the Time table
        self.teacherAvailablity = {}

        self.teacherDetails = [
            {'name':'Karuna','subject':'Math','classes':['11A','12A']},
            {'name':'Reshma','subject':'Math','classes':['11B','12B']},
            {'name':'Moses','subject':'Math','classes':['11C','12C']},

            {'name':'Shafeela','subject':'Chem','classes':['11A','12B']},
            {'name':'Shruthi','subject':'Chem','classes':['11B','12A']},
            {'name':'Krithika','subject':'Chem','classes':['11C','12C']},

            {'name':'Vidhya','subject':'Phy','classes':['11A','12C']},
            {'name':'Anuradha','subject':'Phy','classes':['11B','12A']},
            {'name':'Lida','subject':'Phy','classes':['11C','12B']},

            {'name':'Deepa','subject':'Comp','classes':['11A','12A']},
            {'name':'Judith','subject':'Comp','classes':['11B','11C']},
            {'name':'Shibu','subject':'Comp','classes':['12B','12C']},

            {'name':'Ashwathy','subject':'Bio','classes':['11B','12B']},
            {'name':'Vijitha','subject':'Bio','classes':['11C','12C']},

            {'name':'Anitha','subject':'Eng','classes':['11A','12A','11B','12B']},
            {'name':'Irine','subject':'Eng','classes':['11C','12C']},
        ]

        self.PEPeriods = [
            # Create it so that it assignes according to the period taken py the pe teachers
            {'teacher':'Karthik','subject':'PE','classes':[['11A','11B'],
['11C','11D','11E']],'periodsPerDay':5},
            {'teacher':'Ameen','subject':'PE','classes':[['12A','12B'],
['12C','12D','12E']],'periodsPerDay':5},
        ]

        self.skillSubjectPeriods = [
            {'teachers':
['Ameen','Ashley','Kavitha','Ruskin','Karthik'],'subject':'Skill','grade':11,'periodsPerWeek':3},
            {'teachers':
['Ameen','Ashley','Kavitha','Ruskin','Karthik'],'subject':'Skill','grade':12,'periodsPerWeek':3}
        ]
```

```
        self.practicalPeriods = [
            {'name':'P_Comp','lab':['Comp'],'subjects':['Comp'],'classes':[(['11A'],["Deepa"]),
(['11B','11C'],["Shibu",'Reshma'])],"continuously":2},
            {'name':'P_Bio','lab':['Bio'],'subjects':['Bio'],'classes':[(['11B'],["Ashwathy"]),
(['11C'],["Vijitha"])],"continuously":2},
            {'name':'Prac','lab':['Chem','Phy'],'subjects':['Chem','Phy'],'classes':[(['11A'],
["Shruthi","Anuradha"]),(['11B'],["Shafeela","Vidhya"]),(['11C'],
["Shruthi","Lida"])],"continuously":2},
            {'name':'P_Comp','lab':['Comp'],'subjects':['Comp'],'classes':[(['12A'],["Deepa"]),
(['12B','12C'],["Shibu",'Reshma'])],"continuously":2},
            {'name':'P_Bio','lab':['Bio'],'subjects':['Bio'],'classes':[(['12B'],["Ashwathy"]),
(['12C'],["Vijitha"])],"continuously":2},
            {'name':'Prac','lab':['Chem','Phy'],'subjects':['Chem','Phy'],'classes':[(['12A'],
["Shruthi","Vidhya"]),(['12B'],["Shafeela","Anuradha"]),(['12C'],
["Shruthi","Lida"])],"continuously":2},
        ]

        self.periods = [
            {'subject':'Math','classes':['11A','11B','11C','12A','12B','12C'],'maxPerDay':
[2,1],'noPerWeek':7,'repeatPerWeek':1},
            {'subject':'Chem','classes':['11A','11B','11C','12A','12B','12C'],'maxPerDay':
[2,1],'noPerWeek':7,'repeatPerWeek':1},
            {'subject':'Phy','classes':['11A','11B','11C','12A','12B','12C'],'maxPerDay':
[2,1],'noPerWeek':6,'repeatPerWeek':1},
            {'subject':'Comp','classes':['11A','11B','11C','12A','12B','12C'],'maxPerDay':
[2,1],'noPerWeek':6,'repeatPerWeek':1},
            {'subject':'Eng','classes':['11A','11B','11C','12A','12B','12C'],'maxPerDay':
[2,1],'noPerWeek':6,'repeatPerWeek':1},
        ]

        # self.specialPeriods = [
        #     {'subject':'pe','type':'extra','minPerDay':[1,4],'maxPerDay':
[1,0],'noPerWeek':1,'atSameTime':2},
        # ]


        self.skillDay = [False for x in range(0,noOfDays)]
        self.skillAvailability = [[False for x in range(0,periodsPerDay)] for x in range(0,noOfDays)]
        self.groundAvailability = [[0 for x in range(0,periodsPerDay)] for x in range(0,noOfDays)]
        self.labAvailability = {lab:[[False for x in range(0,periodsPerDay)] for x in
range(0,noOfDays)] for lab in self.labs}

        # print(self.groundAvalablity)

        self.data = {}

        # Initialization of Variables
        self.initiateTeachers()
        self.createEmptyData()

    # Creates the empty data in the beginning
    def createEmptyData(self):
        table = []
        # print(temp)
        for dayNum in range(1,self.noOfDays+1):
            table.append(["" for x in range(self.periodsPerDay)])

        for clas in self.classes:
            for div in range(65,ord(clas['maxSection'])+1):
                self.data[str(clas['grade'])+str(chr(div))] = copy.deepcopy(table)

    def getListClasses(self):
        classes = []
        for clas in self.classes:
            for div in range(65,ord(clas['maxSection'])+1):
                classes.append(str(clas['grade'])+str(chr(div)))
        return classes
```

```
    # Reset the teachers Periods for each teacher for all the days
    def initiateTeachers(self):
        '''
        Create empty data for all the teachers in the self.teachers list
        '''
        for teacher in self.teachers:
            self.teacherAvailablity[teacher] = {}
            for day in self.days:
                self.teacherAvailablity[teacher][day] = [False for x in range(0,self.periodsPerDay)]

    def getSkillAcailablity(self):
        temp = []
        for dayIndex,day in enumerate(self.skillAvailability):
            if not self.skillDay[dayIndex]:
                for periodIndex,isOccupied in enumerate(day):
                    if not isOccupied:
                        temp.append((dayIndex,periodIndex))
        return temp

    def removeOccupiedPE(self,lst,classes,teacher):
        temp = []
        flag = 0
        for item in lst:
            flag = 0
            for sec in classes:
                if self.teacherAvailablity[teacher][self.days[item[0]]][item[1]]:
                    # print(teacher,self.teacherAvailablity[teacher][self.days[item[0]]]
[item[1]],self.days[item[0]],item[1])
                    continue
                if self.data[sec][item[0]][item[1]] == "":
                    flag = 1
            if flag:
                temp.append(item)
        return(temp)

    def getFullGroundAvailablity(self,only = "None"):
        temp = []

        if only == "None":
            for dayIndex,day in enumerate(self.groundAvailability):
                for periodIndex,noOfClasses in enumerate(day):
                    temp.append((dayIndex,periodIndex,noOfClasses))
            return temp

        for dayIndex,day in enumerate(self.groundAvailability):
            for periodIndex,noOfClasses in enumerate(day):
                if only == False and noOfClasses < self.maxClassesInGround:
                    temp.append((dayIndex,periodIndex,noOfClasses))
                elif only == True and noOfClasses == self.maxClassesInGround:
                    temp.append((dayIndex,periodIndex,noOfClasses))
        return temp

    def addGroundPeriod(self,dayIndex,periodIndex,by=1):
        self.groundAvailability[dayIndex][periodIndex] += by

    def setGroundAvailability(self,dayIndex,periodIndex,to = 1):
        self.groundAvailability[dayIndex][periodIndex] = to

    def getLabAvailability(self,lab,check = False):
        temp = []
        for dayIndex,day in enumerate(self.labAvailability[lab]):
            for periodIndex,period in enumerate(day):
                if period == check:
                    temp.append((dayIndex,periodIndex,False))
        return temp

    def removeLabTeacherCoinsiding(self,lst,teacher):
        temp = []
        for item in lst:
            if not self.teacherAvailablity[teacher][self.days[item[0]]][item[1]]:
                temp.append(item)
        return(temp)
```

```
    def removeLabClassCoinsiding(self,lst,clas):
        temp = []
        for item in lst:
            if not self.data[clas][item[0]][item[1]]:
                temp.append(item)
        return(temp)

    def removeLabSuccessive(self,lst,no=2):
        temp = []
        for item in lst:
            if (item[0],item[1]+1,False) in lst:
                temp.append(item)
        return(temp)


    def getAvailablePeriodsOfDay(self,clas,day):
        temp = []
        for index,period in enumerate(self.data[clas][day]):
            if not period:
                temp.append(index)
        return temp

    def getPossibleTeacher(self,clas,subject):
        temp = []
        for teacher in self.teacherDetails:
            if teacher['subject'] == subject and clas in teacher['classes']:
                temp.append(teacher['name'])
        if len(temp)>1:
            raise
        return temp[0]

    def getAvailableTeacher(self,day,period):
        temp = []
        for teacher in self.teachers:
            if not self.teacherAvailablity[teacher][day][period+1]:
                temp.append(teacher)
        return temp

    def getTeacherFreePeriodWeek(self,teacher):
        return self.teacherAvailablity[teacher]

    def getTeacherFreePeriodDay(self,teacher,day):
        return self.teacherAvailablity[teacher][day]

    def isTeacherFree(self,teacher,day,period):
        if self.teacherAvailablity[teacher][day][period+1]:
            return False
        else:
            return True

    def removeTeacherNotAvailable(self,lst,day,teacher):
        temp = []
        for period in lst:
            if not self.teacherAvailablity[teacher][self.days[day]][period]:
                temp.append(period)
        return temp

    def getFreeClassesDay(self,day,period):
        temp = []
        for clas in self.data:
            for subject in self.data[clas][day]:
                if subject in ['PE','Prac','P_Bio','P_Comp']:
                    temp.append(f'{clas},Day:{self.days[day]},Period={period+1}')
        return temp
```

```
    def getFreeContinuesClasses(self,day = None):
        temp = []
        continuousTemp = []

        for clas in self.data:
            if day == None:
                for day in range(self.noOfDays):
                    for period,subject in enumerate(self.data[clas][day]):
                        if subject in ['PE','Prac','P_Bio','P_Comp']:
                            temp.append((day,period,clas))
            else:
                for period,subject in enumerate(self.data[clas][day]):
                        if subject in ['PE','Prac','P_Bio','P_Comp']:
                            temp.append((day,period,clas))

        continuousTemp = []
        for item in temp:
            if (item[0],item[1]+1,item[2]) in temp:
                continuousTemp.append(item)
                continuousTemp.append((item[0],item[1]+1,item[2]))
            continuousTemp.append("")

        return continuousTemp

    # Create Empty files to save it later
    def saveAsCSV(self):
        os.mkdir('Classes')

        for clas in self.classes:
            os.chdir('Classes')
            os.mkdir(str(clas['grade'])+"Grade")
            os.chdir('../')

            for classData in self.data:
                grade = ''.join([i for i in classData if i.isdigit()])

                if int(grade) == int(clas['grade']):
                    filePath = "Classes/"+str(grade)+"Grade/"+classData+".csv"

                    with open(filePath, 'w', newline='') as file:
                        writer = csv.writer(file)

                        header = [classData]
                        for period in range(self.periodsPerDay):
                            header.append(period+1)
                        writer.writerow(header)

                        temp = []
                        for index,day in enumerate(self.data[classData]):
                            temp.append([self.days[index]]+day)

                        row_list = temp
                        writer.writerows(row_list)
    # Create a list and return a list of all classes and sections together in the form of a string in
list, ie.['11A','11B','12A','12B'].
```

Timetable.py

```
import random as rand
from data import *
import copy

class Timetable(Data):
    def __init__(self,periodsPerDay,noOfDays):
        super().__init__(periodsPerDay,noOfDays)

    # Assign skill period for a specific grade together
    def assignSkillPeriods(self):
        classes = self.getListClasses().copy()
        for period in self.skillSubjectPeriods:
            self.skillDay = [False for x in range(0,self.noOfDays)]
            for x in range(period['periodsPerWeek']):
                choosenPeriod = rand.choice(self.getSkillAcailablity())
                for section in self.getListOfSections(period['grade']):
                    self.data[section][choosenPeriod[0]][choosenPeriod[1]] = period['subject']
                    self.skillAvailability[choosenPeriod[0]][choosenPeriod[1]] = True
                    self.skillDay[choosenPeriod[0]] = True
                for teacher in period['teachers']:
                    self.teacherAvailablity[teacher][self.days[choosenPeriod[0]]][choosenPeriod[1]] =
str(period['grade'])+'Skill'

    # Teacher thinky, (Availablity)
    def assignPEPeriod(self):
        """
        Assign PE periods too classes according to the data provided
        """
        for subject in self.PEPeriods:
            for classes in subject['classes']:
                # self.removeOccupiedPE() also removes the Teachers Coinciding
                choosenPeriod =
rand.choice(self.removeOccupiedPE(self.getFullGroundAvailablity(False),classes,subject['teacher']))
                for section in classes:
                    if self.data[section][choosenPeriod[0]][choosenPeriod[1]]:
                        raise Exception("Place Already taken by Skill, if this come then -4hrs by
debugging")
                    self.data[section][choosenPeriod[0]][choosenPeriod[1]] = subject['subject']
                    self.teacherAvailablity[subject['teacher']][self.days[choosenPeriod[0]]]
[choosenPeriod[1]] = section
                    self.addGroundPeriod(choosenPeriod[0],choosenPeriod[1],2)

    def assignLabPeriods(self):
        """
        Assign lab periods for practical classes considering availability of labs, teachers, and
classes.
        """
        for practical in self.practicalPeriods:
            for classes in practical['classes']:
                for lab in practical['lab']:
                    available = self.getLabAvailability(lab)
                    for teacher in classes[1]:
                        available = self.removeLabTeacherCoinsiding(available,teacher)
                    for clas in classes[0]:
                        available = self.removeLabClassCoinsiding(available,clas)
                    available = self.removeLabSuccessive(available,practical['continuously'])
                    choosenPeriod = rand.choice(available)
                for clas in classes[0]:
                    for con in range(practical['continuously']):
                        self.data[clas][choosenPeriod[0]][choosenPeriod[1]+con] = practical['name']
                        for lab in practical['lab']:
                            self.labAvailability[lab][choosenPeriod[0]][choosenPeriod[1]+con] = True
                        for teacher in classes[1]:
                            self.teacherAvailablity[teacher][self.days[choosenPeriod[0]]]
[choosenPeriod[1]+con] = clas
```

```
    # Print the time table along with the classes and days
    def printTimetable(self,plain = False):
        """
        Prints the Generated Time Table.
        """
        for x in self.data:
            print("--",x,"--")
            temp = []
            if not plain:
                for i,y in enumerate(self.data[x]):
                    for z in y:
                        try:
                            temp.append(z['subject'])
                        except:
                            temp.append(z)
                    print(self.days[i],temp,"-",temp.count(""))
                    temp = []
            else:
                for x in self.data:
                    for y in self.data[x]:
                        print(y," -",temp.count(""))
                    print()
            print("\n")

    def printTimetableDict(self):
        print(self.data)



    def getOrdinalsList(self):
        data = self.ordinals[0:self.periodsPerDay]
        return data

    def getDaysList(self):
        data = self.days[0:self.noOfDays]
        return data

    def getData(self):
        # data = {}
        # for index,day in enumerate(self.data['11A']):
        #     data[self.days[index]] = day
        # return data
        return self.data

    # Main Generating Function
    def generateTimetable(self):
        self.assignSpecialPeriods()
        self.assignNormalPeriods()
        # self.getAvailableTeacher("Sun",2)

        # print(self.getAvailableTeacher("Thu",0))

        # print(self.getTeacherFreePeriodDay("Deepa","Sun"))

        # print("Deepa")
        # hours = 0
        # for index,x in enumerate(self.getTeacherFreePeriodWeek("Deepa")):

        #     for y in self.days[index],self.getTeacherFreePeriodWeek("Deepa")[x]:
        #         if y != False:
        #             hours+= 1
        # print(hours,"Hours")

        # for period in self.getFreeContinuesClasses(4):
        #     if not period == "":
        #         print(f'Class: {period[2]}, Day: {self.days[period[0]]}, Period: {period[1]}')
        #     else:
        #         print()

    def updateSQLData(self):
        self.updateOutput(self.data)
```

Home.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?
family=Arvo:ital,wght@0,400;0,700;1,400;1,700&family=Graduate&display=swap" rel="stylesheet">
    <link href="https://fonts.googleapis.com/css2?family=Graduate&display=swap" rel="stylesheet">
    <link href="https://fonts.googleapis.com/css2?
family=Arvo:ital,wght@0,400;0,700;1,400;1,700&family=Graduate&family=Rubik+Mono+One&display=swap"
rel="stylesheet">

    <link rel="stylesheet" href="{{ url_for('static', filename='css/table.css') }}">
    <title>Timetable</title>
</head>
<body>
    <h1 style="text-align: center;">Weekly Timetable</h1>
    {% for class in data %}
        <div>
            <h2 style="text-align: center;">{{class}}</h2>
            <table>
                <thead>
                    <tr>
                        <td>{{class}}</td>
                        {% for ord in table.getOrdinalsList() %}
                            <th>{{ord}}</th>
                        {% endfor %}
                    </tr>
                </thead>
                <tbody>
                    {% for day in data[class] %}
                    <tr>
                        <th>{{ table.days[loop.index0] }}</th>
                        {% for subject in day %}
                            <td>{{subject}}</td>
                        {% endfor %}
                    </tr>
                    {% endfor %}
                </tbody>
            </table>
        </div>
    {% endfor %}
</body>
</html>
```

Landing.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="{{ url_for('static', filename='css/landing.css') }}">
    <link href="https://fonts.googleapis.com/css2?
family=Work+Sans:ital,wght@0,100..900;1,100..900&display=swap" rel="stylesheet">
    <link href="https://fonts.googleapis.com/css2?
family=Nunito:ital,wght@0,200..1000;1,200..1000&family=Work+Sans:ital,wght@0,100..900;1,100..900&display
=swap" rel="stylesheet">
    <link href="https://fonts.googleapis.com/css2?
family=Londrina+Sketch&family=Nunito:ital,wght@0,200..1000;1,200..1000&display=swap" rel="stylesheet">
    <title>TableTimePro</title>
</head>
<body>
    <div id="headerContainer">
        <div id="headerLogo">
        </div>
        <div class="headerButtonsContainer">
            <button class="headerButtons">Know More</button>
            <button class="headerButtons">Hell Naw</button>
            <button class="headerButtons headerButtonsSpecial">TableTime Pro</button>
            <button class="headerButtons">About Us</button>
            <button class="headerButtons">Contact Us</button>
        </div>
        <div class="headerButtonsContainer">
        </div>
    </div>
    <div class="page" style="padding-top:7vh;height:93vh;">
        <div id="headingContainer">
            <h1 class="heading">Effortlessly Organize <br> Your Timetable</h1>
            <p>A web application that helps you effortlessly createand manage your school timetables.
<br> With built in capablities to manage teachers and their classes. This software is especially made
for Indian Schools in Oman.</p>
        </div>
        <button id="launchButton">Generate Timetable</button>
        <img src="{{ url_for('static', filename='assets/background.jpg') }}" alt="backround"
id="background-image">
        <!-- <h4 id="description">Welcome to TableTimePro, a web application that helps you <br> create
and manage your school timetables.</h4> -->
    </div>

    <div class="page">
        <h1>Table TimePro</h1>
        <p>Welcome to TableTimePro, a web application that helps you create and manage your school
timetables.</p>
    </div>
</body>
</html>
```

Landing.css

```css
:root{
    --light-text:hsl(0, 0%, 100%);
    --blue-text:#4d677d;
    --mid-text:#343434ec;
    --text:#1b1b1b;
    --transparent: rgba(0, 0, 0, 0.75);
    --darker:#0d1013;
    --background:#ffffff;
    --med:#1a2128;
    --contrast:#3daee2;
    --mid-contrast:#56b8e6;
    --light-contrast:#5dcad8;
}

*{
    color: var(--text);
    padding: 0px;
    margin: 0px;
    overflow-x: hidden;

    -ms-overflow-style: none;
    scrollbar-width: none;

    font-family: "Work Sans", sans-serif;
    font-optical-sizing: auto;
    font-weight: 400;
    font-style: normal;

    text-align: center;
}

.no-scroll-body::-webkit-scrollbar {
    display: none;
}

#headerContainer{
    display: flex;
    align-items: center;
    justify-content: space-between;

    position: fixed;
    top: 0px;
    right: 0px;
    z-index: 5000;

    padding: 0px 4px;

    width: 100%;
    height: 7vh;
}

.headerButtonsContainer{
    display: flex;
}

.headerButtons{
    background-color: transparent;
    color: var(--mid-text);
    height: 6vh;
    width: 6.4vw;

    cursor: pointer;
    border: none;

    font-family: "Nunito", sans-serif;
    font-optical-sizing: auto;
    font-weight: 500;
    font-size: 0.9vw;

    transition: font-weight 0.1s;
}
```

```css
.headerButtons:hover{
    font-weight: 700;
}

.headerButtonsSpecial{
    color: var(--contrast);
    font-weight: 700;
    width: auto;
    font-size: 1.3vw;
    cursor: pointer;
    transition: font-weight 0.1s;
}

.headerButtonsSpecial:hover{
    font-weight: 800;
}

.page{
    display: flex;
    align-items: center;
    justify-content: center;
    flex-direction:column;

    height: 100vh;
    width: 100vw;
    background-color: var(--background);
}

#headingContainer{
    text-align: center;
}

.heading{
    font-weight: 600;
    font-style: normal;
    font-size: 2.5vw;
    margin-bottom: 1vh;

    font-family: "Nunito", sans-serif;
    font-optical-sizing: auto;
    font-weight: 500;

}

#headingContainer>p{
    font-size: 0.8vw;

    font-family: "Nunito", sans-serif;
    font-optical-sizing: auto;
    font-weight: 500;
    font-size: 0.9vw;
}

#launchButton{
    font-size: 17px;
    height: 8vh;
    width: 15vw;
    font-weight: 500;
    background: var(--contrast);
    color: white;
    border: none;
    position: relative;
    overflow: hidden;
    border-radius: 0.4em;
    cursor: pointer;

    margin-top: 15px;
}

#launchButton:hover{
    background: var(--mid-contrast);
}
```

```css
#launchButton:active{
    background: var(--light-contrast);
}


#description{
    color: var(--blue-text);
    margin-top: 20px;
}

#background-image{
    height:45vh;
    width: 30vw;
}

.text{
    color: var(--text);
}
```

Table.css

```css
*{
    font-family: "Graduate", serif;
    font-weight: 400;
    font-style: normal;
}

table{
    width: 50%;
    border-collapse: collapse;
    margin: 20px auto;
}

th,td{
    border: 1px solid black;
    padding: 10px;
    text-align: center;
    font-family: "Arvo", serif;
    font-weight: 400;
    font-style: normal;
}

th{
    font-family: "Arvo", serif;
    font-weight: 700;
    font-style: normal;

    font-weight: 700;
    font-style: normal;
}

th{
    background-color: #f2f2f2;
}

h1{
    font-family: "Rubik Mono One", monospace;
    font-weight: 400;
    font-style: normal;
}

h2{
    font-family: "Arvo", serif;
    font-weight: 700;
    font-style: normal;
}
```

Bibliography:

[1]. Lutz, M. (2013). *Learning Python* (5th ed.). O'Reilly Media.

[2]. Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python* (2nd ed.). O'Reilly Media.

[3]. MySQL Documentation. (2024). *MySQL 8.0 Reference Manual*. Oracle. Retrieved from https://dev.mysql.com/doc/refman/8.0/en/

[4]. Niemann, K. (2017). *Automating Tasks with Python*. Packt Publishing.

[5]. Kumar, R. (2021). *Database Management Systems with SQL*. Wiley.

[6]. Django Software Foundation. (2024). *Flask vs Django: Choosing the Right Framework*. Retrieved from https://flask.palletsprojects.com/en/2.0.x/

[7]. Python Software Foundation. (2024). *Python Programming Language*. Retrieved from https://www.python.org/

[8]. Anderson, J. (2020). *Timetable Generation Algorithms and Applications*. Journal of Computer Science, 35(3), 102-115.

[9]. Lee, S., & Park, J. (2019). *Optimizing School Timetable Generation with Heuristic Algorithms*. International Journal of Computational Science, 12(4), 220-229.

[10]. Roberts, D. (2018). *A Comparative Study of Scheduling and Timetable Generation Systems*. Procedia Computer Science, 132, 112-118.